

DIAGNÓSTICO DE REPOSITÓRIO • POLIGONAL HUB

Caram3l0 — do README ao Spec-Driven

Análise completa do repositório **poligonalhub/Caram3l0**: estado real do código, práticas de engenharia, e um plano de transição para desenvolvimento orientado por especificações — desenhado para o momento exato em que este projeto está: antes da primeira linha de código de produção.

13 ago 2026 github.com/poligonalhub/Caram3l0 branch: main • HEAD 921d782

licença: CC0 1.0

0

arquivos de
código-fonte

36

commits (35 são
README)

1

contribuidor real

5

arquivos no total

12

meses de vida do
repo

2

arquivos JSON
inválidos

Resumo executivo

O que este repositório realmente é hoje, em uma frase.

Caram3l0 é, neste momento, um manifesto de produto — não um software. O README descreve com clareza uma arquitetura em quatro etapas (farejar, validar, armazenar, latir) para um bot que rastreia editais culturais em João Pessoa e no Norte-Nordeste. É uma ideia bem formada e um problema real. Mas nenhuma dessas etapas existe em código: não há scraper, não há validador, não há integração com Discord/WhatsApp, e os dois únicos arquivos que deveriam conter

integração com Discord/WhatsApp, e os dois únicos arquivos que deveriam conter dados estruturados (`package.json` e `pote/mapacultura.json`) contêm, na verdade, o mesmo parágrafo de texto solto — não são JSON válido.

Isso não é necessariamente ruim: significa que **não existe débito técnico para desfazer**. É a janela mais barata possível para adotar spec-driven development, porque não há código legado competindo com a especificação pela "verdade" do sistema.

Avaliação do projeto

Maturidade, pontos fortes e fracos observados no histórico e nos arquivos.

Estágio 0

Concepção / pré-código

Numa escala Concepção → Especificação → Protótipo → MVP → Produção, o Caram3l0 está no início: visão validada em texto, zero validação em execução. O maior risco não é técnico, é de continuidade — 12 meses de histórico mostram rajadas de edição de README seguidas de longos silêncios.

PONTOS FORTES

- Problema real e bem delimitado: prazos de editais culturais perdidos por falta de monitoramento, com recorte geográfico claro (JP/PB, Norte-Nordeste).
- Metáfora de arquitetura (farejar → validar → pote de razão → latir) é consistente e torna o design do sistema fácil de comunicar a não-engenheiros.
- Lista de fontes de dados iniciais já mapeada (8 portais), o que normalmente é o trabalho de descoberta mais lento em projetos de scraping.
- Decisão de design já registrada no próprio README: um arquivo de scraper

PONTOS FRACOS

- Nenhum código executável; toda a arquitetura descrita é aspiracional, não implementada.
- `package.json` não é JSON — o projeto não roda `npm install` hoje.
- Bus factor de 1: todos os commits são da mesma pessoa (com dois nomes de autor diferentes).
- Sem testes, CI, lint, CONTRIBUTING ou estrutura de pastas — nada disso é grave agora, mas nada foi decidido ainda.

por site, isolando falhas de layout a uma fonte por vez.

Diagnóstico técnico — pontos críticos

Achados ordenados por severidade, com o arquivo e a linha de raciocínio para cada um.

CRÍTICO package.json contém markdown, não JSON

O arquivo na raiz do repo é, byte a byte, um README em miniatura (título, imagem, uma frase de descrição) — não um manifesto npm. Qualquer `npm install`, `npm init` ou tooling padrão de Node falha imediatamente. Isso sinaliza que o arquivo foi criado por engano, provavelmente copiando o conteúdo do README para um novo arquivo sem trocar o conteúdo.

PACKAGE.JSON — CONTEÚDO ATUAL (INVÁLIDO)

```
# caram3lo
![Caram3loV1](\assets\caramelov1.png)
Caram3lo fareja a internet em busca de fontes de financiamento para projetos culturais.
```

CRÍTICO pote/mapacultura.json tem o mesmo problema

Mesmo conteúdo, mesmo bug. O nome do arquivo sugere que deveria ser a saída estruturada do scraper do Mapa da Cultura (uma das 8 fontes listadas no README) — ou seja, é o único artefato do "Pote de Ração" que existe hoje, e está vazio de dado real.

CRÍTICO Nenhuma das 4 etapas da arquitetura está implementada

Scraping (Puppeteer/Cheerio), validação, armazenamento e notificação (Discord/WhatsApp) são mencionados na tabela de stack do README, mas não há uma única dependência declarada, nem um arquivo `.js` no repositório. Antes de qualquer refino de spec, vale confirmar com o mantenedor se existe código em outro branch, fork ou máquina local que não foi enviado — o padrão de commits ("att", merques de dois remotos diferentes) sugere histórico fragmentado.

ALTO README com link markdown quebrado e instrução de instalação truncada

O badge "Poligonal Hub" usa sintaxe de imagem (`![Poligonal Hub](url)`) dentro de uma frase, então renderiza como imagem quebrada em vez de link de texto. O bloco de instalação termina no meio do comando `git clone` , com um link colado incorretamente (`[url](url)` duplicado) — o passo 2 em diante (instalar dependências, configurar variáveis de ambiente, rodar) nunca foi escrito.

ALTO Bus factor 1

Todo o histórico (36 commits) tem o mesmo e-mail de autor sob dois nomes (Laureano. e Tropicaos). Para um projeto que se descreve como "ingressou no movimento de código aberto", não há ainda nenhum sinal de que outra pessoa consiga contribuir sem passar pelo mantenedor original — sem CONTRIBUTING, sem issues abertas, sem templates.

MÉDIO Escolha de licença (CC0 1.0) merece revisão

CC0 dedica a obra ao domínio público — qualquer pessoa pode redistribuir, modificar e até relançar o projeto sob outro nome sem crédito obrigatório. Isso é comum para dados e conteúdo, mas incomum para software com identidade de marca própria (nome "Caram310", mascote, vínculo com "Poligonal Hub"). Se o objetivo é código aberto com atribuição preservada, MIT ou Apache-2.0 cumprem esse papel melhor; CC0 é a escolha certa apenas se o mantenedor realmente não se importa em perder a atribuição.

MÉDIO Ausência total de higiene de projeto Node

Sem CI (GitHub Actions), sem lint/formatter configurado, sem `.env.example` para as credenciais que o projeto vai precisar (webhook do Discord, credenciais de WhatsApp), sem estrutura de pastas (`src/` , `scrapers/` , etc.). O `.gitignore` existe e é um template padrão de Node — é o único artefato de tooling presente hoje.

BAIXO Identidade do projeto ainda instável

O histórico mostra merges vindos de `poligonaldev/Caramel0` e `poligonaldev/caram3l0` antes de convergir para `poligonalhub/Caram3l0`. Nome do produto, organização e grafia (CARAMEL0 → Caram3l0) mudaram ao longo do tempo — normal em fase de concepção, mas vale fixar antes de gerar specs, para não precisar re-nomear módulos depois.

Higienização recomendada antes de especificar

Uma tarde de trabalho que remove todo o atrito acima e deixa o repositório pronto para receber specs.

- ☐ Substituir `package.json` por um manifesto npm real (`name` , `version` , `type: "module"` , `scripts` , licença consistente com o arquivo `LICENSE`).
- ☐ Decidir o propósito real de `pote/mapacultura.json` : é saída de scraper (então deve ser gerado, não versionado, e vai para `.gitignore`) ou é uma seed/fixture (então deve conter um exemplo real da estrutura de dado esperada).
- ☐ Corrigir o link de imagem do README e completar o passo a passo de instalação e execução.
- ☐ Confirmar a licença pretendida (CC0 vs. MIT/Apache-2.0) com base em quanto controle de atribuição o mantenedor quer manter.
- ☐ Criar `CONTRIBUTING.md` e um template de issue/PR mínimo — mesmo sozinho, isso documenta convenções antes que o código exista, o que é mais barato do que depois.
- ☐ Adicionar `.env.example` com as chaves que os módulos de notificação vão precisar (webhook Discord, credenciais WhatsApp), sem valores reais.

Plano de transição para Spec-Driven Development

Por que este é o melhor momento possível, e como estruturar a mudança.

Spec-driven development funciona melhor quando a especificação pode ser a **única fonte de verdade** do sistema — e isso só é totalmente verdade antes de existir código que já divirja dela. Como o Caram3l0 não tem implementação, cada

módulo pode nascer com uma spec aprovada antes da primeira linha, em vez do caminho mais comum (e mais caro) de extrair spec de código legado.

A proposta é tratar as quatro etapas já descritas no README — **Farejando** (coleta), **Rabo Levantado** (validação), **Pote de Ração** (armazenamento) e **Latidos** (notificação) — como quatro módulos com spec própria, mais um quinto módulo de orquestração (o ciclo horário que os conecta). Isso preserva a metáfora que já funciona no README e vira a estrutura de pastas de especificação:

```
/specs |— 01-scrapers-engine # Farejando | |— spec.md # requisitos, critérios de
aceite, fora de escopo | |— design.md # template de scraper, interface comum por
fonte | |— tasks.md # quebra em tarefas implementáveis |— 02-validation-engine #
Rabo Levantado | |— spec.md | |— design.md | |— tasks.md |— 03-storage # Pote de
Ração | |— spec.md | |— design.md | |— tasks.md |— 04-notifications # Latidos |
|— spec.md | |— design.md | |— tasks.md |— 05-scheduler # ciclo horário,
orquestração |— spec.md |— design.md |— tasks.md
```

Cada **spec.md** responde: qual problema este módulo resolve, requisitos funcionais e não-funcionais, critérios de aceite testáveis, e o que fica explicitamente fora de escopo (evita que "Farejando" silenciosamente cresça para dentro de "Rabo Levantado"). Cada **design.md** registra a decisão técnica — por exemplo, a interface comum que todo scraper por site precisa implementar, já sugerida no README. Cada **tasks.md** quebra a spec aprovada em incrementos pequenos o suficiente para implementação assistida, um PR por tarefa.

Regra de governança única: **nada é implementado sem uma spec aprovada por trás**, e todo PR referencia o módulo da spec que está cumprindo. Isso é fácil de manter agora, com um mantenedor, e continua funcionando se o projeto ganhar colaboradores.

Roadmap de implementação assistida

Sequência recomendada — cada fase depende da anterior estar fechada, não apenas iniciada.



0

higienização ~1 tarefa

Os seis itens do checklist acima. Sem isso, qualquer spec escrita já nasce descrevendo um repositório que não existe de fato.

1

Especificação módulo a módulo

Escrever `spec.md` para os 5 módulos, começando pelo Motor de Scraping (é a fonte de dado de tudo mais). Cada spec é revisada e aprovada antes de avançar para o design técnico do mesmo módulo.

2

Fundação do projeto scaffolding

- `package.json` real, estrutura de pastas (`src/scrapers` , `src/validation` , `src/storage` , `src/notifiers`)
- CI mínima: lint + testes vazios rodando a cada push
- Configuração via variáveis de ambiente, nunca segredo versionado

3

Piloto do Motor de Scraping 1 fonte real

Implementar o template comum de scraper e uma única fonte piloto (sugestão: Mapa da Cultura, por ter API/estrutura mais previsível que sites institucionais). Provar o template com uma fonte antes de replicar para as outras 7 evita reescrever a interface depois.

4

Validação + Pote de Ração dados persistidos

Motor de validação (duplicidade, prorrogação, checagem de links, extração de metadados) conectado ao armazenamento estruturado. É aqui que `pote/mapacultura.json` ganha um schema real, versionado como exemplo, não como dado de produção.

5

Latidos Discord → WhatsApp → site

Notificações na ordem de menor para maior custo de integração: webhook do Discord primeiro (mais simples), depois WhatsApp e o site, reaproveitando o mesmo formatter de mensagem descrito na spec do módulo.

6

Orquestração + observabilidade produção

Scheduler horário ligando os 4 módulos com logging mínimo para

selecionar, criando alguns dos 4 módulos, com logging mínimo para diagnosticar quando uma fonte específica quebrar (o próprio README já antecipa esse cenário no design "um scraper por site").

Próximos passos imediatos

O que decidir nesta conversa, antes de abrir o primeiro spec.md.

- ☐ Confirmar se existe código de scraper já escrito em algum outro lugar (máquina local, fork, branch) que deveria ser incorporado antes de especificar do zero.

- ☐ Decidir a licença definitiva (CC0 vs. MIT/Apache-2.0).

- ☐ Escolher qual dos 5 módulos vira o primeiro `spec.md` — recomendação: Motor de Scraping, por ser a dependência de todos os outros.

- ☐ Definir o formato do template de spec a usar (posso gerar um a partir do conteúdo já existente no README, preservando a metáfora *Esquiando/Babo Levantado/Boto da Pação/Latidos*).